

Prácticas de Programación

Práctica 1

Ejercicios

1. Construya un programa ejecutable `nombre` que escriba en la salida estándar dos líneas de texto: la primera con su nombre completo y la segunda con su dirección de correo electrónico en la UDC. El código fuente del programa, escrito en OCaml, debe guardarse en el fichero de texto `nombre.ml`.

El fichero `nombre.ml` debe compilar sin errores con la siguiente orden:

```
ocamlc -o nombre nombre.ml
```

Ejemplo de ejecución:

```
$ ./nombre
Jorge Graña Gil
jorge.grana@udc.es
```

2. Construya un programa ejecutable `pi` que calcule una buena aproximación del número π y muestre el resultado por la salida estándar seguido de un salto de línea. El código fuente del programa, escrito en OCaml, debe guardarse en el fichero de texto `pi.ml`.

El fichero `pi.ml` debe compilar sin errores con la siguiente orden:

```
ocamlc -o pi pi.ml
```

Ejemplo de ejecución:

```
$ ./pi
3.14159265359
```

3. Construya un programa ejecutable `e` que calcule una buena aproximación del número e y muestre el resultado por la salida estándar seguido de un salto de línea. El código fuente del programa, escrito en OCaml, debe guardarse en el fichero `e.ml`. En este ejercicio, el código fuente no puede contener ningún punto y coma.

El fichero `e.ml` debe compilar sin errores con la siguiente orden:

```
ocamlc -o e e.ml
```

Ejemplo de ejecución:

```
$ ./e
2.71828182846
```

4. Se trata ahora de analizar la serie de expresiones escritas (presuntamente) en OCaml incluidas en el fichero `expresiones.pdf` que se proporciona junto con este mismo enunciado. Para ello, abriremos el compilador interactivo de OCaml y, con cada expresión del fichero, haremos lo siguiente:

- La escribimos en el fichero de texto `expr.ml` utilizando un editor de textos.

- Seguidamente, intentaremos predecir la información que proporcionaría el compilador interactivo de OCaml (ocaml) sobre su compilación y ejecución, y lo escribimos como comentario en la línea siguiente, procurando usar la misma notación que ocaml. En OCaml, los comentarios comienzan con (*) y terminan con *).
- Por último, copiamos la expresión en el terminal en el que tengamos abierto el compilador interactivo de OCaml y comprobamos el resultado. Si no es el previsto, lo corregimos e intentamos razonar por qué y en qué nos hemos equivocado.
- Para toda expresión que produzca un error:
 - La escribiremos en el fichero de texto como comentario.
 - Indicaremos, también como comentario, el tipo de error (léxico, sintáctico, de tipo o de ejecución) y la causa del mismo.
- Usaremos el manual del lenguaje para averiguar el significado de los operadores y funciones que desconozcamos. El apéndice "Index of values" del manual contiene una relación completa de los operadores y valores predefinidos en el lenguaje.
- Es importante poner como comentario todo aquello que se pide explícitamente que se escriba así, porque el fichero `expr.ml` debe compilar (aunque obviamente si se genera el correspondiente programa ejecutable, éste no tendrá ningún efecto "visible", porque el fichero `expresiones.pdf` no contiene instrucciones de entrada/salida).

El fichero `expr.ml` debe compilar sin errores con la siguiente orden:

```
ocamlc -c expr.ml
```

Nota Importante: Cuando se solicite la entrega de esta práctica, cada alumno deberá enviar únicamente los ficheros `nombre.ml`, `pi.ml`, `e.ml` y `expr.ml`.

Más adelante se darán indicaciones más precisas sobre la tarea de entrega y sobre el mecanismo de envío de los ficheros.

Sea muy cuidadoso a la hora de crear los ficheros y respete los nombres indicados. En particular, fíjese que todos estos nombres sólo contienen letras en minúsculas y puntos. Además, todos los ficheros deben compilar sin errores con las órdenes anteriormente citadas.

Práctica 2

Ejercicios

1. Se trata de analizar la serie de frases escritas (presuntamente) en OCaml incluidas en el fichero `frases.pdf` que se proporciona junto con este mismo enunciado. Para ello, abriremos el compilador interactivo de OCaml y, con cada frase del fichero, haremos lo siguiente:
 - La escribimos en el fichero de texto `frases.ml` utilizando un editor de textos.
 - Seguidamente, intentaremos predecir la información que proporcionaría el compilador interactivo de OCaml (ocaml) sobre su compilación y ejecución, y lo escribimos como comentario en la línea siguiente, procurando usar la misma notación que ocaml (que distingue claramente expresiones de definiciones). En OCaml, los comentarios comienzan con (*) y terminan con *).

- Por último, copiamos la frase en el terminal en el que tengamos abierto el compilador interactivo de OCaml y comprobamos el resultado. Si no es el previsto, lo corregimos e intentamos razonar por qué y en qué nos hemos equivocado.
- Para toda frase que produzca un error:
 - La escribiremos en el fichero de texto como comentario.
 - Indicaremos, también como comentario, el tipo de error (léxico, sintáctico, de tipo o de ejecución) y la causa del mismo.
- Usaremos el manual del lenguaje para averiguar el significado de los operadores y funciones que desconozcamos. El apéndice "Index of values" del manual contiene una relación completa de los operadores y valores predefinidos en el lenguaje.
- Es importante poner como comentario todo aquello que se pide explícitamente que se escriba así, porque el fichero `frases.ml` debe compilar (aunque obviamente si se genera el correspondiente programa ejecutable, éste no tendrá ningún efecto "visible", porque el fichero `frases.pdf` no contiene instrucciones de entrada/salida).

El fichero `frases.ml` debe compilar sin errores con la siguiente orden:

```
ocamlc -c frases.ml
```

Nota Importante: Cuando se solicite la entrega de esta práctica, cada alumno deberá enviar únicamente el fichero `frases.ml`.

Más adelante se darán indicaciones más precisas sobre la tarea de entrega y sobre el mecanismo de envío de los ficheros.

Sea muy cuidadoso a la hora de crear el fichero y respete el nombre indicado. En particular, fíjese que este nombre sólo contienen letras en minúsculas. Además, el fichero debe compilar sin errores con la orden anteriormente citada.

Práctica 3

Ejercicios

- Complete el archivo `def.ml`, proporcionado con este enunciado, con el código OCaml necesario para definir los siguientes valores (cuando se trate de funciones, deben utilizarse expresiones lambda con forma `function -> ...`):
 - Un valor `pi` de tipo `float` que sea una buena aproximación del número π .
 - Un valor `e` de tipo `float` que sea una buena aproximación del número e .
 - Un valor `max_int_f` de tipo `float` que sea una buena aproximación, como float, de `max_int`.
 - Un valor `perimeter` de tipo `float -> float` que haga corresponder a cada número no negativo el perímetro de la circunferencia que tenga como radio ese número (no importa lo que suceda con los negativos).
 - Un valor `area` de tipo `float -> float` que haga corresponder a cada número no negativo el área del círculo que tenga como radio ese número (no importa lo que suceda con los negativos).
 - Un valor `next_char` de tipo `char -> char` que haga corresponder a cada valor de tipo char el que ocupa la siguiente posición en la tabla ascii.

- Un valor `abs_f` de tipo `float -> float` que haga corresponder a cada número su valor absoluto.
- Una función `odd` de tipo `int -> bool` de modo que, al aplicarla a cualquier entero, el resultado que devuelva indique si el entero es impar. Intente que la definición sea lo más concisa posible (sugerencia: intente evitar el uso de expresiones if-then-else innecesarias).
- Un valor `next_5_mult` de tipo `int -> int` que haga corresponder a cada entero el menor múltiplo de 5 que sea mayor que él.
- Un valor `is_letter` de tipo `char -> bool` que devuelva true en los caracteres de la a a la z (tanto mayúsculas como minúsculas) y false en los demás. A estos efectos, sólo consideraremos como "letras" los caracteres del alfabeto inglés (es decir, quedan excluidas la ñ, la ç, las letras con tilde, etc.).
- Y por último, redefina la función `string_of_bool` de tipo `bool -> string`, de modo que devuelva (adecuadamente) los valores "verdadero" o "falso" (puede hacerse una definición por casos o utilizarse una expresión if-then-else).

El archivo `def.ml` debe compilar sin errores con el archivo de interfaz `def.mli` proporcionado, mediante la orden:

```
ocamlc -c def.mli def.ml
```

2. Las definiciones de función en las que se utiliza una expresión lambda con una sola regla pueden abreviarse siguiendo el siguiente esquema: en vez de escribir

```
let <f> = function <x> -> <e>
```

podemos escribir:

```
let <f> <x> = <e>
```

Analice con cuidado esta abreviatura y reescriba, en un archivo con nombre `def_a.ml`, las definiciones del ejercicio 1, utilizando esta abreviatura. En este ejercicio, por tanto, está prohibida la palabra reservada `function`.

Dado que el archivo `def_a.ml` debe contener las mismas definiciones que `def.ml`, si copia la interfaz `def.mli` a otro archivo `def_a.mli`, puede compilar el nuevo archivo con su interfaz para comprobar que contiene todas las definiciones requeridas. En este caso, la orden de compilación sería:

```
ocamlc -c def_a.mli def_a.ml
```

3. Añadiendo la palabra reservada `rec` a continuación de `let` es posible escribir en OCaml definiciones recursivas de funciones. Intente predecir, y luego compruebe con el compilador interactivo, qué sucede al compilar y ejecutar las siguientes frases:

```
let rec factorial = function 0 -> 1 | n -> n * factorial (n-1);;
factorial 0 + factorial 1 + factorial 2;;
factorial 10;;
factorial 100;;
factorial (-1);;
```

En un archivo con nombre `funciones.ml` defina en OCaml las siguientes funciones recursivas:

- `sum_to` de tipo `int -> int`, de modo que `sum_to n` devuelva la suma de todos los naturales desde 0 hasta n (incluido).

- `exp_2` de tipo `int -> int`, de modo que, para cualquier $n \geq 0$, `exp_2 n` devuelva el valor de 2^n .
- `num_cifras` de tipo `int -> int`, de modo que `num_cifras n` devuelva el número de cifras de la representación decimal de n (el signo no cuenta como cifra).
- `sum_cifras` de tipo `int -> int`, de modo que, `sum_cifras n` devuelva la suma de las cifras correspondientes a la representación decimal de n .

Ejemplo de comportamiento:

```
# sum_to 10;;
- : int = 55
# exp_2 9;;
- : int = 512
# num_cifras (-1999);;
- : int = 4
# sum_cifras 129;;
- : int = 12
```

Puede comprobar que el archivo `funciones.ml` contiene todas las definiciones requeridas utilizando la interfaz `funciones.mli` que se proporciona.

4. **Ejercicio opcional.** Recuerde que la evaluación de la aplicación de funciones en OCaml es eager y analice con cuidado la evaluación de las siguientes expresiones:

```
let pi = 2. *. asin 1.0 in pi *. pi;;
(function pi -> pi *. pi) (2. *. asin 1.0);;
```

Debería llegar a la conclusión de que ambas se evalúan exactamente igual. En general la frase:

```
let <id> = <e2> in <e1>
```

será equivalente a:

```
(function <id> -> <e1>) <e2>
```

Utilice esta relación para eliminar todas las definiciones locales en el código OCaml del ejercicio 1 de la práctica 2. Para ello, copie el archivo `frases.ml` escrito durante la realización de ese ejercicio en un archivo `frases_2.ml` y, a continuación, sustituya cada una de las expresiones que contenga una definición local, siguiendo el esquema anterior.

Reescriba también las definiciones que contengan expresiones lambda, utilizando la abreviatura explicada en el ejercicio 2 de esta misma práctica, de modo que, al final, no aparezca la palabra `function` dentro de ninguna definición.

5. **Ejercicio opcional.** Si `<b>` es una expresión correcta de tipo `bool` en OCaml y `<e1>` y `<e2>` son expresiones del mismo tipo, entonces:

```
if <b> then <e1> else <e2>
```

se evalúa igual que:

```
(function true -> <e1> | false -> <e2>) <b>
```

Analice esta equivalencia y utilícela para reescribir el siguiente código OCaml sin utilizar frases `if-then-else` (las palabras `if`, `then` y `else` están prohibidas):

```
if x > y then "x is greater" else "y is greater";;  
if x > 0 then x else -x;;  
if x > 0 then x else if y > 0 then y else 0;;  
if x > y then if x > z then x else z else if y > z then y else z;;
```

Escriba el código equivalente en un archivo con nombre `if_then_else.ml`.

Práctica 4

Ejercicios

1. Considere la función de $\mathbb{N} \rightarrow \mathbb{N}$, que podría aproximarse en OCaml con la siguiente definición:

```
let f n = if n mod 2 = 0 then n / 2 else 3 * n + 1
```

Según la Conjetura de Collatz, si partimos de cualquier número positivo y aplicamos repetidamente esta función, llegaremos inexorablemente al 1.

```
let rec check n =  
  n = 1 || check (f n)
```

En un archivo con nombre `collatz.ml`, defina una función `check_to` de tipo `int -> bool`, de modo que `check_to n` devuelva `true` después de comprobar la conjetura en todos los naturales hasta el `n` (incluido).

```
# check_to 1_000_000;;  
- : bool = true
```

Llamaremos "órbita" de un número al camino obtenido al aplicar repetidamente, desde ese número, la función `f` hasta llegar al 1. Añada al mismo archivo la definición de una función recursiva `orbit` de tipo `int -> string` tal que, para cualquier `n > 0`, devuelva un string con la representación de su órbita (los valores separados por una coma y un espacio, sin espacio final):

```
# orbit 13;;  
- : string = "13, 40, 20, 10, 5, 16, 8, 4, 2, 1"
```

Añada al mismo archivo la definición de una función recursiva `length` de tipo `int -> int` tal que devuelva el número de valores que contiene la órbita de `n`.

```
# length 13;;  
- : int = 10
```

Añada al mismo archivo la definición de una función recursiva `top` de tipo `int -> int` tal que sea el valor más alto alcanzado en la órbita de `n`.

```
# top 13;;  
- : int = 40
```

2. **Ejercicio opcional.** Defina directamente (de modo recursivo, sin usar las funciones `length` y `top`) una función `length_and_top` de tipo `int -> int * int` tal que, para cada entero `n`, devuelva un par indicando la longitud de su órbita y su valor máximo, sin recorrer dos veces la órbita. Incorpórela a un archivo de nombre `collatz_plus.ml`.
3. Euclides describió el algoritmo para calcular el MCD de dos números. Utilice la versión original de Euclides (por restas sucesivas) para implementar de modo recursivo en OCaml una función `mcd` de tipo `int * int -> int`.
- ```
mcd (12, 20);;
- : int = 4
```
- Una versión mucho más eficiente del algoritmo consiste en reemplazar, en cada paso, el mayor de ambos números por el resto de su división por el menor. Defina una función `mcd'` de tipo `int * int -> int` aprovechando esta mejora. Ambas definiciones deben incluirse en un archivo con nombre `mcd.ml`.
4. **Ejercicio opcional.** Defina una función `mcd_pasos` de tipo `int * int -> int * int` que calcule el MCD según el algoritmo de Euclides optimizado y devuelva un par: el valor del MCD y el número de pasos necesarios para calcularlo. Guarde esta función en un archivo con nombre `mcd_plus.ml`.

## Práctica 5

### Ejercicios

1. Estudie la definición recursiva ineficiente de Fibonacci:

```
let rec fib n =
 if n <= 2 then 1
 else fib (n-1) + fib (n-2)
```

Utilizando esta definición, construya un programa ejecutable `fibto`, de modo que su ejecución con un argumento `n` (mayor que 0) muestre por la salida estándar todos los términos de la sucesión de Fibonacci menores que `n`. El código fuente de este programa debe escribirse en un archivo de nombre `fibto.ml` sin utilizar ni el punto y coma (;) ni el doble punto y coma (;;). El programa debe compilar correctamente con la orden:

```
$ ocamlc -o fibto fibto.ml
```

2. Intente realizar una definición más eficiente de la función `fib` y construya con ella un nuevo programa `fastfibto` substituyendo la definición original de `fib` (hágalo en un archivo `fastfibto.ml`).
3. **Ejercicio opcional.** En un archivo `strg.ml`, defina una función `str3_of_int` de tipo `int -> string` tal que, para cualquier `int n >= 0`, devuelva la representación en base 3 de `n`.

```
str3_of_int 5;;
- : string = "12"
```

4. **Ejercicio opcional.** Defina también una función `int_of_str3` de tipo `string -> int` de modo que si `s` representa un número natural en base 3, devuelva su valor correspondiente módulo  $2^{\text{Sys.int\_size}}$ .

5. **Ejercicio opcional.** Defina dos nuevas funciones `strg_of_int` de tipo `int -> int -> string` e `int_of_strg` de tipo `int -> string -> int` que generalicen las funciones de los dos ejercicios anteriores para cualquier base entre 2 y 9, siendo el primer parámetro la base.

## Práctica 6

### Ejercicios

1. Analice el código OCaml de las Torres de Hanoi proporcionado en el enunciado. Reconstruya las definiciones perdidas de las funciones `mueve` y `otro`, de modo que las funciones se comporten exactamente como en los ejemplos. Complete el archivo `hanoi.ml` suministrado.
2. Añada al archivo `hanoi.ml` la definición de una función `n_hanoi_mov` de tipo `int -> int -> int -> int -> int * int` de modo que `n_hanoi_mov n nd ori des` devuelva el  $n$ -ésimo movimiento de la secuencia de movimientos necesarios para resolver el problema con `nd` discos desde el poste `ori` al poste `des`.
3. Implemente una versión más eficiente de la función de comprobar si un número es primo: `is_prime2` de tipo `int -> bool` en el fichero de nombre `prime.ml`.
4. En el mismo fichero `prime.ml` escriba una función `goldbach` de tipo `int -> int * int` que, dado un número entero positivo par `n`, encuentre dos números primos `p1` y `p2` tales que `n = p1 + p2`.
5. **Ejercicio opcional.** Realice las siguientes tareas en un fichero de texto `ej65.ml`:
  - Defina las funciones de curificación y descurificación: `curry` y `uncurry`.
  - Defina la composición de funciones curificada: `comp`.
  - Defina funciones polimórficas con tipos específicos como se indica en el enunciado.

## Práctica 7

### Ejercicios

1. Redefina en un archivo `fact.ml` la función factorial de tal manera que, cuando se aplique sobre números negativos, provoque la excepción `Invalid_argument "fact"`. Asegúrese de que la comprobación se realice sólo una vez y no en cada llamada recursiva.
2. Escriba en un archivo con nombre `myList.ml` las definiciones necesarias para satisfacer la interfaz `myList.mli` proporcionada. Deberá implementar versiones recursivas normales y terminales (marcadas con un apóstrofe, p. ej. `append'`) de las funciones estándar del módulo `List`, utilizando únicamente pattern-matching y sin recurrir al módulo `List` original ni al operador de concatenación (`@`).

## Práctica 8

### Ejercicios

1. Utilizando la función `fold_left` del módulo `List`, defina las funciones descritas en el archivo `folding.mli` proporcionado. En este ejercicio no está permitido utilizar ninguna otra de las funciones del módulo `List` (aparte de `fold_left`). Tampoco se permite usar recursividad (palabra reservada `rec`). Guarde las definiciones en un archivo con nombre `folding.ml`.

2. Defina las funciones descritas en el archivo `listing.mli`. Ahora sí puede utilizar las funciones del módulo `List` (tales como `init`, `map`, `filter`, etc.). Sólo puede utilizarse recursividad terminal. Las definiciones deben guardarse en un archivo con nombre `listing.ml`.

## Práctica 9

### Ejercicios

1. Analice el problema de la ineficiencia de la implementación de `concat` con `List.fold_left` y `List.append`. Explique por qué sucede esto como comentario en un archivo con nombre `concat.ml` e incluya en el mismo archivo dos nuevas definiciones:
  - Una con nombre `concat'` escrita utilizando `List.fold_right`.
  - Y otra con nombre `concat''` que sea recursiva terminal sin utilizar `List.append`.
2. Añada al archivo `concat.ml` una función `sublists` de tipo `'a list -> 'a list list` que, dada una lista, devuelva la lista de todas sus sublists respetando el orden relativo de los elementos de la lista original.
3. Escriba en un archivo de nombre `tail.ml` versiones terminales de las siguientes funciones: `front`, `compress` y `fold_right`.
4. **Ejercicio opcional.** Identifique cuáles de las funciones del ejercicio anterior pueden ser implementadas con la transformación "Tail Modulo Constructor" (TMC) y escriba en un archivo `tail2.ml` nuevas definiciones para ellas utilizando esta funcionalidad.

## Práctica 10

### Ejercicios

1. Un preso intenta escapar cruzando un patio de dimensiones `m x n` baldosas, evitando colisionar con robots de vigilancia que se mueven en cada paso en una de las cuatro direcciones (North, South, East, West) y rebotan en los bordes. El preso se mueve de baldosa a baldosa en vertical u horizontal. Implemente una función `tour` de tipo:

```
int -> int -> ((int * int) * move) list -> move list
```

que calcule una ruta de escape válida desde (1,1) hasta (m,n), o lance la excepción `Not_found` si no hay ruta. Realice su implementación en el archivo `tour.ml`.

2. **Ejercicio opcional.** Implemente una función `shortest` con el mismo tipo que `tour`, pero que calcule la lista de movimientos correspondiente a la ruta de escape más corta. Realice su implementación en un archivo con nombre `shortest.ml`.

## Práctica 12

### Ejercicios

1. **Ejercicio opcional.** Modifique el archivo `simpleC.ml` proporcionado para extender el intérprete de Simple C de modo que dé soporte a nuevas características que se detallan en los archivos del material descargado. El

tipo `statement` y la gestión del contexto deben actualizarse en consecuencia para interactuar con la memoria del programa.